



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Lab Experiment – 03

Water Jug Problem

Aim

To solve the Water Jug Problem using the State Space Search algorithm.

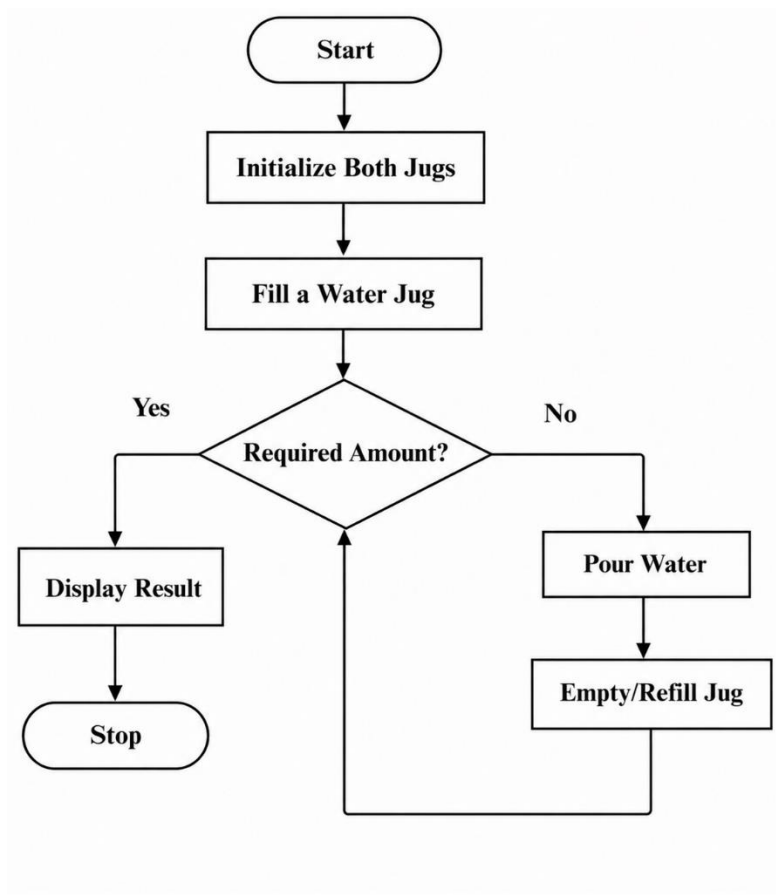
Objective

To obtain the required amount of water in one of the jugs by performing fill, empty, and pour operations.

Algorithm

1. Start with both jugs empty.
2. Fill one of the jugs completely.
3. Pour water from one jug to the other until the receiving jug is full or the source jug is empty.
4. Fill or empty the jugs whenever necessary.
5. Repeat the above steps until the required quantity of water is obtained.
6. Display the sequence of states.
7. Stop.

FlowChart:



Code:

```
from collections import deque
```

```
def water_jug(cap1, cap2, target):
```

```
    visited = set()
```

```
    queue = deque()
```

```
    queue.append((0, 0))
```

```
    visited.add((0, 0))
```

```
    while queue:
```

```
        x, y = queue.popleft()
```

```
        print((x, y))
```

```
if x == target or y == target:
```

```
    print("Target Reached")
```

```
    return
```

```
next_states = [
```

```
    (cap1, y),          # Fill Jug1
```

```
    (x, cap2),          # Fill Jug2
```

```
    (0, y),             # Empty Jug1
```

```
    (x, 0),             # Empty Jug2
```

```
    (x - min(x, cap2 - y), y + min(x, cap2 - y)), # Jug1 -> Jug2
```

```
    (x + min(y, cap1 - x), y - min(y, cap1 - x)) # Jug2 -> Jug1
```

```
]
```

```
for state in next_states:
```

```
    if state not in visited:
```

```
        visited.add(state)
```

```
        queue.append(state)
```

```
water_jug(4, 3, 2)
```

Output:

```
C: > Users > Asus > OneDrive > Desktop > Python Codes > Experiment 3.py > ...
1  from collections import deque
2
3  def water_jug(cap1, cap2, target):
4      visited = set()
5      queue = deque()
6
7      queue.append((0, 0))
8      visited.add((0, 0))
9
10     while queue:
11         x, y = queue.popleft()
12         print((x, y))
13
14         if x == target or y == target:
15             print("Target Reached")
16             return
17
18         next_states = [
19             (cap1, y),          # Fill Jug1
20             (x, cap2),         # Fill Jug2
21             (0, y),            # Empty Jug1
22             (x, 0),            # Empty Jug2
23             (x - min(x, cap2 - y), y + min(x, cap2 - y)), # Jug1 -> Jug2
24             (x + min(y, cap1 - x), y - min(y, cap1 - x)) # Jug2 -> Jug1
25         ]
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(4, 3)
(1, 3)
(3, 0)
(1, 0)
(3, 3)
(0, 1)
(4, 2)
Target Reached
```

Result

The Water Jug Problem was solved successfully using the State Space Search algorithm.

Conclusion

The program successfully finds the required amount of water by applying valid fill, empty, and pour operations between the two jugs.